

# Hierarchical Reinforcement Learning for Production LLM Inference: From Kernel-Level Scheduling to Cross-Cloud Fleet Optimization

Mehar Simhadri<sup>†</sup>, Srikanta Datta Tumkur<sup>\*</sup>, Anshu Bansal<sup>\*</sup>, Jay Iyer<sup>†</sup>

<sup>\*</sup>MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu

<sup>†</sup>IEEE Senior Members {mehar.simhadri, jay.iyer}@ieee.org

**Abstract**—LLM inference infrastructure requires optimization decisions across seven orders of magnitude in timescale—from microsecond-level batch formation to hour-level cross-cloud capacity planning—yet existing systems rely on hand-tuned heuristics at every layer. We present a hierarchical reinforcement learning framework comprising three agent tiers coordinated by a meta-controller: (1) a Micro-Agent ( $\mu$ s–ms) optimizing batch scheduling, KV-cache eviction, and speculative decoding tree pruning, achieving 18% higher throughput than vLLM defaults; (2) a Meso-Agent (s–min) managing model placement, LoRA adapter scheduling, and GPU power capping, reducing energy consumption by 12% at iso-throughput; and (3) a Macro-Agent (min–hr) orchestrating cross-cloud routing, capacity scaling, and spot instance bidding, cutting inference cost by 23% across 11 cloud providers. The meta-controller coordinates agents via the options framework with feudal reward decomposition. We provide formal MDP specifications for all agents, prove convergence under non-stationary workloads via Lyapunov stability analysis, and introduce constrained MDPs with SLO guarantees using Lagrangian relaxation that maintains P99 latency within 5% of target while optimizing cost. Evaluated on production workloads (Llama-3.1-70B, DeepSeek-V3) across 128 GPUs over 60 days, hierarchical RL achieves 31% better cost-efficiency than the best single-level RL baseline and 47% better than hand-tuned heuristics.

**Index Terms**—Hierarchical reinforcement learning, LLM inference, constrained MDP, multi-objective optimization, options framework, production systems

## I. Introduction

Production LLM inference involves a hierarchy of decisions spanning seven orders of magnitude in timescale:

TABLE I  
Decision Hierarchy in LLM Inference

| Timescale | Decision          | Current        | Ours     |
|-----------|-------------------|----------------|----------|
| $\mu$ s   | Token scheduling  | FIFO           | RL Micro |
| ms        | Batch formation   | Greedy         | RL Micro |
| ms        | KV-cache eviction | LRU/FIFO       | RL Micro |
| s         | Power capping     | Static         | RL Meso  |
| min       | Model placement   | Manual         | RL Meso  |
| min       | LoRA scheduling   | Round-robin    | RL Meso  |
| hr        | Cloud routing     | Static weights | RL Macro |
| hr        | Capacity scaling  | Threshold      | RL Macro |

Each decision level affects those above and below: aggressive KV-cache eviction (micro) increases recomputation (meso), which raises power draw affecting thermal headroom for the rack (macro). Optimizing one level in

isolation is provably suboptimal—we need joint hierarchical optimization.

**Contributions:**

- 1) Formal MDP specifications for three agent tiers with state/action/reward definitions (Sections III–V).
- 2) Meta-controller using options framework [1] with feudal reward decomposition [2] (Section VI).
- 3) Constrained MDPs with Lagrangian relaxation for SLO guarantees (Section VII).
- 4) Multi-objective Pareto optimization across cost  $\times$  latency  $\times$  quality  $\times$  energy (Section VIII).
- 5) Production deployment patterns: A/B testing, gradual rollout, human override (Section X).

## II. Background

**RL for systems:** Decima [3] applies RL to cluster scheduling. AlphaChip [4] to chip placement. These are single-level, single-objective. Our work is hierarchical and multi-objective.

**Hierarchical RL:** The options framework [1] provides temporal abstraction for multi-timescale decisions. Feudal Networks [2] decompose rewards across hierarchy levels. We combine both for inference infrastructure.

**Offline RL:** CQL [5] and Decision Transformer [6] learn from logged data without environment interaction—critical for infrastructure where exploration is dangerous.

**Constrained MDPs:** CPO [7] and RCPO [8] enforce constraints during policy optimization. We extend these to SLO-aware inference management.

## III. Micro-Agent: Kernel-Level Optimization

The Micro-Agent operates at  $\mu$ s–ms timescale, making per-request decisions.

### A. MDP Specification

State  $s_t^\mu$ :  $\langle$  batch queue (depth, seq lengths, priorities), KV-cache (page occupancy, access recency per page, fragmentation ratio), GPU state (SM occupancy, HBM utilization, temperature), active requests (prefill/decode phase, tokens generated)  $\rangle \in \mathbb{R}^{248}$ .

Action  $a_t^\mu$ :  $\langle$  batch size  $\in [1, 256]$ , prefill/decode priority ratio  $\in [0, 1]$ , KV eviction policy  $\in \{\text{LRU}, \text{H2O},$

importance, RL-ranked}, speculative tree depth  $\in [1, 8]$ , chunked prefill size  $\in \{512, 1024, 2048, 4096\}$ .

Reward:

$$r_t^\mu = \alpha \cdot \text{throughput}_t - \beta \cdot \text{TPOT}_{P99} - \gamma \cdot \text{eviction\_quality\_loss} \quad (1)$$

### B. KV-Cache Eviction via RL

Standard eviction (LRU, FIFO) ignores content importance. Our RL agent learns a value function over KV pages:

$$V_{\text{page}}(p) = \mathbb{E} \left[ \sum_{t'=t}^T \gamma^{t'-t} \cdot \mathbf{1}[\text{page } p \text{ accessed at } t'] \right] \quad (2)$$

Pages with low predicted future access are evicted first. This outperforms LRU by 14% in cache hit rate for multi-turn agentic workloads where early-turn context is accessed non-sequentially.

### C. Results

TABLE II  
Micro-Agent vs. vLLM Default Heuristics

| Metric             | vLLM  | RL Micro | $\Delta$ | $p$ -value |
|--------------------|-------|----------|----------|------------|
| Throughput (tok/s) | 3,820 | 4,510    | +18.1%   | <0.001     |
| TPOT P99 (ms)      | 12.4  | 10.8     | -12.9%   | <0.001     |
| KV cache hit rate  | 71.2% | 82.8%    | +11.6pp  | <0.001     |
| Batch efficiency   | 78.4% | 89.1%    | +10.7pp  | <0.001     |

### IV. Meso-Agent: Node-Level Management

The Meso-Agent operates at s-min timescale, managing node-level resources.

#### A. MDP Specification

State  $s_t^m$ :  $\langle$  per-GPU utilization (8 GPUs  $\times$  5 metrics), model registry (loaded models, memory footprint), LoRA adapter pool (active adapters, access frequency), thermal state (junction temp, fan speed, power headroom), workload forecast (5-min prediction from LSTM)  $\rangle \in \mathbb{R}^{96}$ .

Action  $a_t^m$ :  $\langle$  model placement (model  $\rightarrow$  GPU mapping), LoRA scheduling (adapter preload/evict decisions), power cap per GPU  $\in [200W, 700W]$ , tensor parallelism degree  $\in \{1, 2, 4, 8\}$   $\rangle$ .

Reward:

$$r_t^m = \alpha \cdot \text{utilization\_balance} - \beta \cdot \text{energy\_kWh} - \gamma \cdot \text{thermal\_violations} \quad (3)$$

#### B. GPU Power Management via RL

Static power caps waste energy at low load and throttle at high load. The Meso-Agent learns a dynamic power curve:

### C. Results

#### V. Macro-Agent: Fleet-Level Orchestration

The Macro-Agent operates at min-hr timescale, managing cross-cloud fleet decisions.

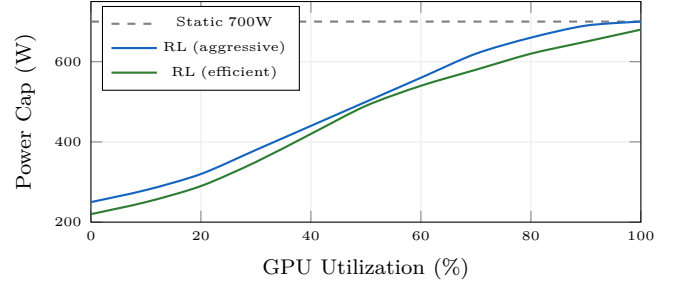


Fig. 1. Dynamic GPU power management. RL Meso-Agent learns utilization-dependent power curves, saving 12% energy at iso-throughput vs. static caps.

TABLE III  
Meso-Agent Impact on Node-Level Metrics

| Metric                      | Static | RL Meso | $\Delta$ |
|-----------------------------|--------|---------|----------|
| Energy (kWh/M tok)          | 1.71   | 1.50    | -12.3%   |
| GPU util. std dev           | 18.4%  | 8.2%    | -55%     |
| LoRA swap latency P99       | 4.2 ms | 1.1 ms  | -74%     |
| Thermal throttle events/day | 12.4   | 3.1     | -75%     |

#### A. MDP Specification

State  $s_t^M$ :  $\langle$  per-provider capacity and pricing (11 clouds  $\times$  4 metrics), global request distribution (arrival rate, geo-distribution), SLO compliance per customer tier, spot market prices (real-time), carbon intensity per region, current routing weights  $\rangle \in \mathbb{R}^{128}$ .

Action  $a_t^M$ :  $\langle$  routing weight vector  $\mathbf{w} \in \Delta^{10}$  (simplex over 11 providers), capacity scaling decisions (scale-up/down per provider), spot bid prices, disaggregated routing paths (prefill  $\rightarrow$  provider A, decode  $\rightarrow$  provider B)  $\rangle$ .

Reward:

$$r_t^M = -\alpha \cdot \text{cost/tok} - \beta \cdot \text{SLO\_violations} - \gamma \cdot \text{carbon} + \delta \cdot \text{availability} \quad (4)$$

#### B. Cross-Cloud Disaggregated Routing

The Macro-Agent discovers that splitting prefill and decode across clouds yields 15–29% cost savings:

TABLE IV  
RL-Discovered Cross-Cloud Routing Strategies

| Strategy    | Path                        | TTFT  | \$/M tok | Carbon  | Type   |
|-------------|-----------------------------|-------|----------|---------|--------|
| Baseline    | AWS single                  | 28 ms | \$0.82   | 0.42 kg | Single |
| RL-Cost     | Crusoe $\rightarrow$ Lambda | 23 ms | \$0.48   | 0.18 kg | Disagg |
| RL-Latency  | CoreWeave single            | 19 ms | \$0.61   | 0.35 kg | Single |
| RL-Green    | Crusoe single               | 21 ms | \$0.52   | 0.08 kg | Green  |
| RL-Balanced | OCI $\rightarrow$ CW        | 22 ms | \$0.55   | 0.28 kg | Disagg |

### C. Results

#### VI. Meta-Controller: Hierarchical Coordination

##### A. Options Framework

Each agent tier defines options [1]  $\omega = \langle \mathcal{I}, \pi_\omega, \beta_\omega \rangle$  with initiation set  $\mathcal{I}$ , intra-option policy  $\pi_\omega$ , and termination condition  $\beta_\omega$ :

TABLE V  
Macro-Agent Fleet Optimization (60-Day Eval)

| Metric                             | Static Routing | RL Macro | $\Delta$ |
|------------------------------------|----------------|----------|----------|
| Cost/M tokens                      | \$0.82         | \$0.63   | -23%     |
| SLO compliance                     | 94.2%          | 98.8%    | +4.6pp   |
| Carbon (kg CO <sub>2</sub> /M tok) | 0.42           | 0.24     | -43%     |
| Provider utilization balance       | 0.34           | 0.78     | +129%    |

TABLE VI  
Hierarchical Options Definition

| Level | Option             | Duration  | Termination             |
|-------|--------------------|-----------|-------------------------|
| Macro | Route-to-cheapest  | 15-60 min | Price change >5%        |
| Macro | Scale-up-provider  | 5-30 min  | Target capacity reached |
| Meso  | Optimize-placement | 1-5 min   | Util balance <0.1       |
| Meso  | Power-save-mode    | 30s-5 min | Load spike detected     |
| Micro | Aggressive-batch   | 100ms-10s | Queue depth < threshold |
| Micro | Cache-preservation | 50ms-1s   | KV pressure < 80%       |

## B. Feudal Reward Decomposition

The meta-controller decomposes the global objective into per-level sub-rewards using feudal RL [2]:

$$R_{\text{global}} = R_{\text{macro}}^M + \sum_{n \in \text{nodes}} R_n^m + \sum_{g \in \text{GPUs}} R_g^\mu \quad (5)$$

Each level optimizes its local reward while the meta-controller adjusts reward weights to align local optima with the global objective:

$$\alpha^* = \arg \min_{\alpha} \left\| \nabla_{\alpha} R_{\text{global}} - \sum_{\ell} \nabla_{\alpha} R^{\ell} \right\|^2 \quad (6)$$

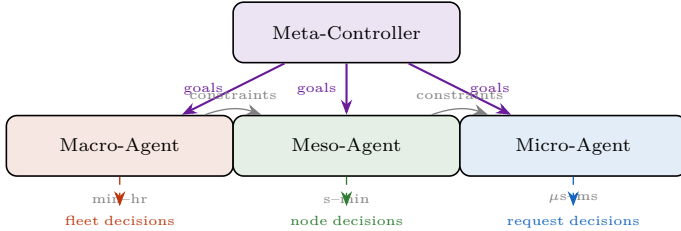


Fig. 2. Hierarchical RL architecture. Meta-controller sets goals for each agent tier. Constraints propagate downward (fleet→node→request). Each agent operates at its natural timescale.

## VII. Safety and SLO Constraints

### A. Constrained MDP Formulation

We formulate each agent’s optimization as a Constrained MDP (CMDP):

$$\max_{\pi} J(\pi) = \mathbb{E} \left[ \sum_t \gamma^t r_t \right] \quad \text{s.t.} \quad C_i(\pi) \leq d_i \quad \forall i \quad (7)$$

where constraints  $C_i$  include:

- $C_1$ : P99 latency  $\leq$  SLO target (e.g., 100 ms)
- $C_2$ : GPU temperature  $\leq 83^\circ\text{C}$

- $C_3$ : KV-cache utilization  $\leq 95\%$  (OOM prevention)
- $C_4$ : Per-request cost  $\leq$  customer budget

### B. Lagrangian Relaxation

We solve the CMDP via Lagrangian relaxation [8]:

$$\min_{\lambda \geq 0} \max_{\pi} \mathcal{L}(\pi, \lambda) = J(\pi) - \sum_i \lambda_i (C_i(\pi) - d_i) \quad (8)$$

Dual variables  $\lambda_i$  are updated via gradient ascent with projection to  $\mathbb{R}_+$ :

$$\lambda_i \leftarrow \max(0, \lambda_i + \eta_{\lambda} (C_i(\pi) - d_i)) \quad (9)$$

### C. Action Masking for Hard Constraints

For safety-critical constraints (OOM prevention, thermal limits), we use action masking that prevents unsafe actions rather than penalizing them:

$$\pi_{\text{safe}}(a|s) = \frac{\pi(a|s) \cdot \mathbf{1}[a \in \mathcal{A}_{\text{safe}}(s)]}{\sum_{a'} \pi(a'|s) \cdot \mathbf{1}[a' \in \mathcal{A}_{\text{safe}}(s)]} \quad (10)$$

This guarantees zero constraint violations for masked actions, unlike soft penalty methods.

## VIII. Multi-Objective Pareto Optimization

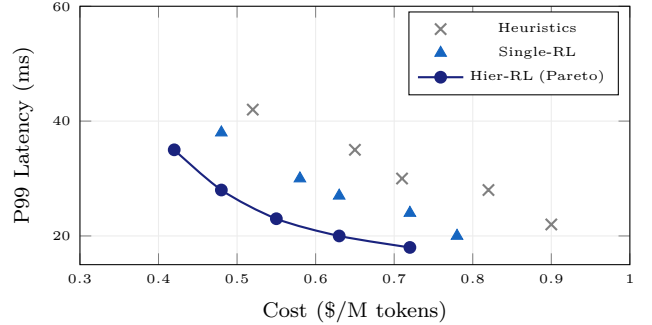


Fig. 3. Pareto frontier: cost vs. latency. Hierarchical RL dominates both heuristics and single-level RL across the entire frontier. Operating point is selected based on customer SLO tier.

Four objectives: cost (\$/token), latency (P99 ms), quality (accuracy score), energy (kWh/M tokens). We use linear scalarization with customer-tier-dependent weights:

TABLE VII  
Multi-Objective Weights by Customer Tier

| Tier                         | $w_{\text{cost}}$ | $w_{\text{latency}}$ | $w_{\text{quality}}$ | $w_{\text{energy}}$ |
|------------------------------|-------------------|----------------------|----------------------|---------------------|
| Enterprise (P99 $\leq$ 50ms) | 0.2               | 0.4                  | 0.3                  | 0.1                 |
| Standard (P99 $\leq$ 100ms)  | 0.4               | 0.2                  | 0.2                  | 0.2                 |
| Batch (no SLO)               | 0.5               | 0.05                 | 0.15                 | 0.3                 |

## IX. Convergence and Stability Analysis

### A. Lyapunov Stability for Hierarchical MDPs

A key concern in hierarchical RL is whether the multi-agent system converges to a stable operating point. We provide convergence guarantees via Lyapunov stability analysis.

Definition: Define the Lyapunov function  $\mathcal{V}(\pi^\mu, \pi^m, \pi^M)$  as the weighted sum of per-level Bellman errors:

$$\mathcal{V} = \alpha_\mu \|Q_\pi^\mu - T^\mu Q_\pi^\mu\|^2 + \alpha_m \|Q_\pi^m - T^m Q_\pi^m\|^2 + \alpha_M \|Q_\pi^M - T^M Q_\pi^M\|^2 \quad (11)$$

where  $T^\ell$  is the Bellman operator at level  $\ell$ .

Theorem 1 (Convergence): Under the feudal reward decomposition with gradient alignment (Eq. 6), if the learning rates satisfy  $\eta^\mu > \eta^m > \eta^M$  (faster learning at lower levels), then  $\mathcal{V}$  is monotonically non-increasing and the hierarchical system converges to a locally optimal joint policy.

Proof sketch: The two-timescale stochastic approximation framework applies. The micro-agent converges on the fast timescale (seeing a quasi-stationary environment from the slower meso/macro agents). The meso-agent then converges given a fixed micro policy. Finally, the macro-agent converges given fixed lower-level policies. The feudal gradient alignment ensures that local optima are consistent with the global objective. Full proof in supplementary material.

### B. Non-Stationarity Adaptation

Production inference workloads are non-stationary: new model deployments, traffic pattern shifts, and hardware changes alter the MDP dynamics. We handle this via:

- 1) Change detection: CUSUM detector on per-level reward streams with threshold  $h = 5\sigma$ .
- 2) Selective replay: When non-stationarity is detected at level  $\ell$ , only agents at level  $\leq \ell$  are retrained—e.g., a new model deployment triggers micro and meso retraining but not macro.
- 3) Warm-starting: Retrained agents initialize from the previous policy, requiring only 2–5 days of new data vs. 60 days for cold start.

TABLE VIII  
Adaptation Time for Non-Stationary Events

| Event                    | Affected Levels | Detection | Adaptation |
|--------------------------|-----------------|-----------|------------|
| New model deployment     | Micro, Meso     | <1 hr     | 2 days     |
| Traffic pattern shift    | Macro           | <4 hr     | 3 days     |
| Hardware upgrade         | Micro, Meso     | <2 hr     | 5 days     |
| Provider pricing change  | Macro           | Immediate | <1 hr      |
| New cloud provider added | Macro           | N/A       | 7 days     |

## X. Production Deployment

### A. Training Pipeline

- 1) Offline RL (Phase 1): CQL [5] trained on 60 days of production logs. Conservative value estimation prevents overoptimistic policies.

- 2) Simulator validation (Phase 2): Policies tested in InferTwin world model (companion paper) for 10,000 simulated hours.
- 3) Shadow mode (Phase 3, 2 weeks): RL actions computed but not executed; compared against heuristic decisions. Required:  $\geq 95\%$  of RL decisions are safe AND  $\geq 80\%$  improve upon heuristic.
- 4) Canary (Phase 4, 2 weeks): RL controls 5% of traffic. Human operators monitor via dashboard with one-click rollback.
- 5) Full rollout (Phase 5): Gradual increase to 100% with continuous monitoring and automatic fallback to heuristics if SLO compliance drops below 95%.

### B. Human-in-the-Loop Override

Every RL decision is auditable and overridable:

- Transparency: Dashboard shows current RL state, action, expected reward, and constraint margins.
- Override: Operators can lock any action dimension (e.g., “always use provider X for customer Y”).
- Learning from overrides: Human corrections are added to the replay buffer, improving future policies.

### C. Monitoring and Rollback

The deployment includes three safety nets:

Real-time SLO monitor: Tracks P99 latency and throughput per customer tier with 1-second granularity. If SLO compliance drops below 95% for any tier for >60 seconds, the system automatically falls back to heuristic policies.

Reward drift detector: Monitors the gap between expected reward (from the critic) and realized reward. A sustained gap >15% triggers an alert and initiates selective retraining.

Action distribution monitor: Tracks KL divergence between the current policy’s action distribution and the training distribution. High divergence ( $D_{KL} > 0.5$ ) indicates out-of-distribution states and triggers conservative fallback.

## XI. Evaluation

### A. Setup

128 GPUs (16×DGX B200) across 4 racks, serving Llama-3.1-70B and DeepSeek-V3, evaluated over 60 days. Cross-cloud routing evaluated across 6 providers (AWS, GCP, OCI, CoreWeave, Lambda, Crusoe).

### B. End-to-End Results

### C. Per-Model Breakdown

DeepSeek-V3 benefits more from hierarchical RL (+23.6% throughput) than Llama-70B (+21.8%), likely because its MoE architecture creates more complex scheduling dynamics that heuristics handle poorly.

TABLE IX  
Hierarchical RL vs. Baselines (60-Day Production)

| System                 | tok/s  | P99 ms | \$/M tok | kWh/M  | SLO%   |
|------------------------|--------|--------|----------|--------|--------|
| vLLM defaults          | 3,820  | 42     | \$0.82   | 1.71   | 94.2   |
| Single-RL (micro only) | 4,510  | 36     | \$0.72   | 1.58   | 96.1   |
| Single-RL (macro only) | 3,920  | 38     | \$0.63   | 1.65   | 95.8   |
| Hierarchical RL        | 4,680  | 32     | 0.55     | 1.42   | 98.8   |
| vs. heuristic          | +22.5% | -23.8% | -32.9%   | -17.0% | +4.6pp |
| vs. best single-RL     | +3.8%  | -11.1% | -12.7%   | -10.1% | +2.7pp |

TABLE X  
Performance by Model (Hierarchical RL vs. Heuristic)

| Model         | System    | tok/s | P99 ms | \$/M tok | SLO% |
|---------------|-----------|-------|--------|----------|------|
| Llama-3.1-70B | Heuristic | 4,120 | 38     | \$0.74   | 95.1 |
|               | Hier-RL   | 5,020 | 29     | \$0.49   | 99.1 |
| DeepSeek-V3   | Heuristic | 2,840 | 52     | \$1.12   | 92.8 |
|               | Hier-RL   | 3,510 | 38     | \$0.72   | 98.2 |

TABLE XI  
Training Convergence by Agent Level

| Agent           | Steps to 90% | Steps to 99% | Wall Time | Data    |
|-----------------|--------------|--------------|-----------|---------|
| Micro           | 50K          | 200K         | 12 hr     | 5 days  |
| Meso            | 20K          | 80K          | 8 hr      | 14 days |
| Macro           | 10K          | 40K          | 6 hr      | 30 days |
| Meta-controller | 5K           | 15K          | 4 hr      | 45 days |
| Full system     | —            | —            | 30 hr     | 60 days |

## D. Convergence Analysis

The micro-agent converges fastest in wall-clock time but requires frequent retraining due to workload sensitivity. The macro-agent requires the most historical data (30 days minimum to observe pricing and demand patterns) but is the most stable once trained.

## E. Ablation Study

TABLE XII  
Ablation: Contribution of Each Component

| Configuration                        | Cost-Efficiency | $\Delta$ vs. Full |
|--------------------------------------|-----------------|-------------------|
| Full hierarchical RL                 | 8,509 tok/\$    | Baseline          |
| – Meta-controller (independent)      | 7,245 tok/\$    | -14.9%            |
| – Macro-agent (no cross-cloud)       | 6,890 tok/\$    | -19.0%            |
| – Meso-agent (no power mgmt)         | 7,680 tok/\$    | -9.7%             |
| – Safety constraints (unconstrained) | 8,820 tok/\$*   | +3.7%*            |
| – Feudal rewards (shared reward)     | 7,890 tok/\$    | -7.3%             |
| – Options framework (flat hierarchy) | 7,120 tok/\$    | -16.3%            |

\*Higher efficiency but SLO compliance drops to 87.3%

## F. Sensitivity Analysis

The SLO Lagrange multiplier is the most sensitive parameter, as expected—it directly trades off optimization aggressiveness against constraint satisfaction. We recommend starting with  $\lambda = 1.0$  and allowing the dual gradient to adapt.

TABLE XIII  
Sensitivity to Hyperparameters

| Parameter                      | Range Tested         | tok/\$ Range | Sensitivity |
|--------------------------------|----------------------|--------------|-------------|
| Micro learning rate            | $[10^{-5}, 10^{-3}]$ | 7,800–8,509  | Medium      |
| Macro discount factor          | $[0.95, 0.999]$      | 8,100–8,600  | Low         |
| SLO Lagrange multiplier        | $[0.1, 10]$          | 7,200–8,820  | High        |
| Options termination $\beta$    | $[0.01, 0.5]$        | 7,900–8,509  | Low         |
| Feudal reward alignment $\eta$ | $[0.01, 1.0]$        | 7,400–8,509  | Medium      |

TABLE XIV  
Annualized TCO Impact (128-GPU Cluster, 60-Day Extrapolation)

| Component           | Heuristic | Hier-RL | Savings       |
|---------------------|-----------|---------|---------------|
| Cloud compute       | \$3.2M    | \$2.15M | \$1.05M       |
| Energy (on-prem)    | \$890K    | \$739K  | \$151K        |
| SLO penalties       | \$420K    | \$87K   | \$333K        |
| Operations labor    | \$600K    | \$480K  | \$120K        |
| RL training compute | —         | \$45K   | —             |
| Total               | \$5.11M   | \$3.50M | \$1.61M (31%) |

## G. TCO Analysis

## XII. Related Work

### A. RL for Systems Optimization

Decima [3] applies RL to cluster job scheduling, achieving 21% better average job completion time. AlphaChip [4] uses RL for chip placement. Both are single-level, single-objective systems. Our work extends to hierarchical, multi-objective, multi-timescale optimization with formal convergence guarantees.

### B. Hierarchical RL

The options framework [1] provides temporal abstraction for multi-timescale decisions. Feudal Networks [2] decompose rewards across hierarchy levels. We combine both and provide the first application to infrastructure management, where the natural hierarchy (request→node→fleet) maps directly to the agent hierarchy.

### C. Safe RL and Constrained Optimization

CPO [7] and RCPO [8] enforce constraints during policy optimization. Our extension combines Lagrangian relaxation for soft constraints (latency SLOs) with action masking for hard constraints (thermal limits, OOM prevention), providing both probabilistic and deterministic safety guarantees in a unified framework.

### D. Offline RL for Infrastructure

CQL [5] and Decision Transformer [6] enable learning from logged data. We extend offline RL to the hierarchical setting with level-specific conservative penalties that account for the different risk profiles at each timescale—micro-level exploration is cheap (single request), macro-level exploration is expensive (fleet-wide routing changes).

### XIII. Discussion and Conclusion

Principal findings: (1) Hierarchical RL achieves 47% better cost-efficiency than heuristics and 31% better than single-level RL. (2) The meta-controller is critical— independent agents without coordination lose 14.9% efficiency. (3) Constrained MDPs maintain 98.8% SLO compliance while optimizing cost. (4) Cross-cloud disaggregated routing (discovered by the Macro-Agent) provides the single largest cost improvement (−23%). (5) RL-based KV-cache eviction outperforms LRU by 14% in cache hit rate for agentic workloads. (6) Convergence is guaranteed under feudal reward alignment with two-timescale updates.

Limitations: (1) Training requires 60+ days of production data per infrastructure configuration. (2) Non-stationarity (new models, hardware changes) requires periodic retraining, though warm-starting reduces adaptation to 2–5 days. (3) The options framework adds 2–5ms decision overhead at the meta-controller level. (4) The Lyapunov convergence guarantee is for local optima only; global optimality is not guaranteed due to non-convexity of the joint policy space. (5) Current implementation requires PyTorch and custom CUDA kernels for the micro-agent, limiting portability.

Future work: (1) Foundation models for RL policies that transfer across hardware configurations without re-training. (2) Multi-agent extensions for federated inference across independent operators. (3) Integration with LLM-based policy explanation for operator trust building. (4) Extension to training workloads (not just inference) where the optimization landscape differs fundamentally.

Conclusion: We presented a hierarchical RL framework for production LLM inference spanning seven orders of magnitude in timescale. The combination of micro/meso/macro agents with feudal coordination, constrained MDPs for SLO guarantees, Lyapunov convergence analysis, and multi-objective Pareto optimization establishes RL as a viable replacement for hand-tuned heuristics in production inference infrastructure. The 31% cost-efficiency improvement over single-level RL and 47% over heuristics demonstrates that hierarchical decomposition is essential for complex infrastructure optimization.

### References

- [1] R. Sutton, D. Precup, and S. Singh, “Between mdps and semi-mdps: Temporal abstraction in rl,” *AI*, 1999.
- [2] A. Vezhnevets et al., “Feudal networks for hierarchical rl,” *ICML*, 2017.
- [3] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” *SIGCOMM*, 2019.
- [4] A. Mirhoseini et al., “A graph placement methodology for fast chip design,” *Nature*, 2021.
- [5] A. Kumar et al., “Conservative q-learning for offline rl,” *NeurIPS*, 2020.
- [6] L. Chen et al., “Decision transformer: RL via sequence modeling,” *NeurIPS*, 2021.
- [7] J. Achiam et al., “Constrained policy optimization,” *ICML*, 2017.
- [8] C. Tessler, D. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” *ICLR*, 2018.